ClearShift V1 Team Contributor Guide

How to understand an issue, build safely with GitHub, AI, an IDE and Antigravity, and deliver a reviewable pull request

ClearShift Project Management Office

19 October 2026

Contents

1. How to use this guide **3**
Important baseline-control note . . . 3
What "done" means . . . 3

2. ClearShift in plain English **3**
The business problem . . . 4
System shape . . . 4

3. Baseline, scope and decisions that must not drift **4**
3.1 Technical baseline . . . 4
3.2 V1 boundaries . . . 4
3.3 Immutable data rules . . . 5

4. Team map and ownership boundaries **5**
4.1 Four module roles . . . 6
4.2 Team 11 and Workstream 12 . . . 6

5. Issue anatomy and how to select the next issue **7**
5.1 Read the entire issue . . . 7
5.2 Check dependencies and readiness . . . 7
5.3 Size and split . . . 7

6. The eight-step delivery workflow **7**
Step 1 - Claim and prepare . . . 7
Step 2 - Inspect before editing . . . 7
Step 3 - Produce a plan before code . . . 8
Step 4 - Implement the smallest end-to-end slice . . . 8
Step 5 - Verify incrementally . . . 8
Step 6 - Inspect the diff like a reviewer . . . 8
Step 7 - Open a clear PR . . . 8
Step 8 - Review, fix, merge and close . . . 8
Worked example - taking one issue from assignment to PR . . . 8

7. Git and GitHub: the working rules **9**
7.1 Concepts in one minute . . . 9
7.2 Labels and status . . . 9
7.3 Branch naming . . . 9
7.4 Commits . . . 9
7.5 Issue ID vs GitHub issue number . . . 10

8. Local environment and repository discovery **10**
8.1 First inspection . . . 10
8.2 Typical setup sequence . . . 10
8.3 Discover exact check commands . . . 10
8.4 If setup fails . . . 11

9. Using AI effectively without surrendering engineering judgment **11**
9.1 Use AI in stages . . . 11
9.2 Minimum context to give the AI . . . 11
9.3 Prompt rules that improve results . . . 11

10. Google Antigravity: safe, repeatable workflow **12**
10.1 Before the first task . . . 12
10.2 Use Rules for permanent constraints, Skills for repeatable tasks . . . 12
10.3 How to use the agent panel . . . 12
10.4 Parallel agents and worktrees . . . 12
10.5 Never approve these blindly . . . 12

11. Implementation standards by technical area **13**
11.1 Database and Drizzle . . . 13
11.2 Fastify APIs, Zod and OpenAPI . . . 13

11.3 Frontend React + TypeScript + Vite . . . 13
11.4 Redis Streams, transactional outbox and events . . . 13
11.5 BullMQ jobs and batch processing . . . 14
11.6 Security and sensitive data . . . 14

12. Migration Kit rules every team must understand 14
12.1 Row-accounting model . . . 14
12.2 Dry run vs real load . . . 14
12.3 Clean/map rules . . . 14
12.4 Handling a bad batch . . . 15

13. Testing: prove behavior, not just code execution 15
13.1 Test pyramid . . . 15
13.2 Required negative cases . . . 15
13.3 Reconciliation acceptance . . . 15
13.4 Evidence to attach to a PR . . . 15

14. Creating a high-quality pull request 15
14.1 Keep the PR reviewable . . . 15
14.2 Before opening the PR . . . 16
14.3 PR title . . . 16
14.4 PR body must answer . . . 16
14.5 Link the issue correctly . . . 16
14.6 Reviewer assignment and merge . . . 16
14.7 Push and create the PR - exact sequence . . . 16

15. Reviewing another person's pull request 17
Reviewer checklist . . . 17

16. Debugging, conflicts and recovery 18
16.1 When a test fails . . . 18
16.2 When merge conflicts happen . . . 18
16.3 When you discover a design flaw . . . 18

17. Daily work, sprint routines and escalation 18
Daily update (2-3 minutes) . . . 18
Meetings and coordination . . . 18
Escalation rules . . . 18

18. Module-specific cheat sheets 19
Key cross-module master data boundaries . . . 20

19. Ready-to-copy AI prompts 20
Prompt A - Understand; do not edit . . . 20
Prompt B - Approved implementation plan . . . 20
Prompt C - Implement one scoped issue . . . 21
Prompt D - Independent review of my diff . . . 21
Prompt E - Prepare the PR packet . . . 21

20. Checklists and templates 21
Contributor checklist before coding . . . 21
Contributor checklist before PR . . . 21
Maintainer checklist before merge . . . 22
Mini status comment template . . . 22

21. Common failure patterns 22

22. First-day checklist 22

23. Reference links 23

1. How to use this guide

Audience: All module engineers, Platform/Migration Kit engineers, QA, integration engineers, team leads and PMO.

Goal: Give every contributor one repeatable way to take an assigned issue from understanding to merged, tested code without bypassing the architecture or relying blindly on AI.

Rule zero: The issue is a contract, not just a task title. Read its full body, confirm its dependencies, deliver its acceptance criteria, show evidence, and obtain review before it is merged.

Read Sections 2-8 before starting your first issue. Keep Sections 9-15 open while working. Use the module cheat sheet in Section 18 to remember your domain's non-negotiable rules. The companion `AI_Prompt_Playbook.md` contains shorter copy/paste prompts. The package also includes a Pull Request template, an Antigravity workspace rule and an Antigravity issue-execution skill.

This guide explains the working method. The following sources define the project requirements:

1. **The live GitHub issue assigned to you**: the current work item, GitHub issue number, status, assignee, acceptance criteria, dependencies and discussion.
2. **ClearShift V1 30-Issue Standardization Plan**: the 30-slot work standard, role ownership, sprint allocation, technical baseline and V1 boundaries.
3. **Project Plan and the CRM, ERP and Migration Kit SDS documents**: product scope, domain rules, APIs, data model and release gates.
4. **Repository source code and automated tests**: how a decision is already implemented. Code is evidence, not authority to violate an approved specification.

Important baseline-control note

The files supplied with this request include an older issue-catalog snapshot whose validation record counts 148 detailed issue specifications, while the standardization plan sets a target of 360 specifications. You have stated that 360 issues have now been created. This guide therefore assumes the **live GitHub backlog is the operational assignment source**. Do not create duplicate issues from the older 148-item file. If a live issue conflicts with the approved standardization plan or SDS, stop and ask the team lead plus Integration Lead to resolve the conflict in writing before coding.

What "done" means

An issue is not done because an AI generated code, your editor shows no red underline, or the page appears to work once. It is done only when:

- its dependencies are complete;
- every acceptance criterion has a test or other named piece of evidence;
- relevant automated tests pass locally and in CI;
- the code follows the shared architecture and ownership boundaries;
- the PR has the required human review and all review conversations are resolved;
- contracts, schema/data dictionary, docs, fixtures and screenshots are updated where applicable;
- no secret, real customer/employee data, unrelated refactor or generated junk is included;
- the change is merged through the repository's approved process.

2. ClearShift in plain English

ClearShift helps move business data from old files or systems into a new ERP or CRM while cleaning and validating the records and proving what happened to every source row.

The **Migration Kit is the main product**. ClearShift CRM and ClearShift ERP are small, controlled target systems used to demonstrate that the Kit can load realistic business data correctly. V1 uses synthetic data only.

A migration is more than "read a CSV and insert rows." The intended pipeline is:

1. **Extract** - intake CSV or XLSX and preserve original row values.
2. **Profile** - report blanks, formats, types, ranges and potential duplicates.
3. **Clean** - apply registered, deterministic cleanup rules.
4. **Map** - map source columns and old values to target fields and enums.
5. **Dry run** - show intended outcomes without changing target business rows or target import receipts.
6. **Load** - write accepted rows in bounded batches using stable idempotency keys and durable import receipts.

7. **Validate** - check domain rules and cross-module references.
8. **Reconcile** - prove the row outcomes exactly account for the parsed source.

The seven core engine steps named in the SDS group dry-run and load as the loading step; this guide separates dry run for operational clarity. Follow the issue's exact pipeline acceptance criteria and the SDS; do not create a second pipeline.

The business problem

A customer, product, vendor, invoice or employee may be represented differently in different source files. A migration can fail through lost rows, duplicate records, broken relationships, invalid business states, unsafe data exposure or incorrect financial totals. ClearShift's purpose is to control those failures and produce evidence that they did not silently happen.

System shape

- **Business modules** own their domain schemas, validation, APIs, domain screens and module-specific target adapters.
- **Platform / Hub** owns common toolchain, app bootstrap, shared UI, identity/authentication and authorization helpers, common telemetry, Golden ID services and the event relay infrastructure.
- **Migration Kit** owns generic source intake, profiling, cleaning/mapping engine, exact duplicate workflow, batch execution, review/reporting UI and reconciliation engine.
- **Integration Lead / integration workstream** controls API and event contracts, data ownership, integration waves and cross-module dependency decisions.
- **QA / PMO / Release** defines and verifies test evidence, defect triage, gate checklists and the go/no-go decision.

3. Baseline, scope and decisions that must not drift

3.1 Technical baseline

Use the version pinned in the repository lockfiles and approved plan. The approved planning baseline is:

Area	Standard
Runtime	Node.js 24 LTS, exact patch pinned by the repo
Language/package tools	Strict TypeScript; pnpm; Turborepo monorepo
Backend	Fastify 5
Validation/API contract	Zod and OpenAPI
Database/ORM	PostgreSQL 18.6 and Drizzle
Batch jobs	BullMQ workers
Domain event delivery	Redis Streams, backed by the PostgreSQL transactional outbox
Frontend	React + TypeScript + Vite; common shared component library and design tokens
Tests	Vitest, Fastify injection/API contract checks, React Testing Library, Playwright and k6

The versions above are a baseline, not permission to update dependencies independently. If `package.json`, lockfiles, Docker images or CI specify a pinned patch, use that exact version. Version changes require an approved dependency/architecture decision and a dedicated PR with compatibility evidence.

Never introduce an alternative queue or framework as a convenience. Temporal.io, ARQ, Celery, a second Redis queue, a second API framework or per-team design system are not approved substitutions. If the live issue or an existing branch seems to require a different stack, do not silently choose: raise an architecture decision request.

3.2 V1 boundaries

V1 is the Must-have product slice through Gate G4 at the end of Week 8. V1 must demonstrate safe intake, profile, deterministic clean/map rules, a dry run, loading, validation and exact reconciliation for synthetic data.

The source intake contract in the standardization plan is:

- CSV using comma or semicolon delimiter, UTF-8;
- XLSX first sheet only;
- maximum input size 50 MB;
- streamed/bounded processing; do not buffer an unbounded file in memory;
- no source database connectors and no cloud file upload in V1;
- batch size 500 rows;
- row-level errors may be accumulated, but stop the run when failures exceed 10% of attempted rows;
- stable idempotency key = `batch ID + source row + target entity/schema version`;
- a retry must reuse the same key;
- dry run must not mutate target business rows or target import receipts;
- each accepted target-row write and its import receipt commit in the same local PostgreSQL transaction.

V1 excludes fuzzy matching/scoring, real customer or employee data, old-database connectors, outbound marketing delivery and a compensating rollback saga. If a V1 load fails, stop safely, preserve evidence and receipts, keep retries bounded, and recover according to the issue. Do not quietly build rollback as part of V1.

3.3 Immutable data rules

Every parsed source row receives exactly one terminal outcome:

- `loaded`
- `rejected`
- `merged`
- `skipped`

The reconciliation must enforce:

```
source = loaded + rejected + merged + skipped
```

```
unaccounted = 0
```

A skipped row needs an explicit allowed reason and an auditable pre-load decision. Ignoring a source column does **not** mean the row was skipped. No source row may disappear between parse, stage, load and report.

Additional invariants:

- Raw source values are preserved for audit; normalized values are separate derived values.
- Unknown/uncertain duplicates must not be merged automatically. V1 uses deterministic normalized exact keys; any human approval path specified by the issue must be explicit and audited.
- Target-side writes are idempotent across retries.
- One owning module owns each master-data type. Consumers read through approved APIs/projections; they do not create a second mutable truth.
- Material domain mutations write their audit/outbox records atomically where required by the shared helper.
- Financial calculations use decimal/numeric arithmetic, not binary floating point.
- Do not log secrets, raw sensitive payloads, bank details or government identity values.
- Fixtures and demos use synthetic data only.

4. Team map and ownership boundaries

Workstream	Main responsibility	Owns	Must not duplicate
Group 1 / CRM-1	Contacts, companies and leads	Customer/contact master data and its APIs	Other modules' customer master writes
Group 2 / CRM-2	Deals, pipeline and quotes	Sales opportunity and quote domain	Product master or sales-order ownership
Group 3 / CRM-3	Support tickets and activities	Support case/activity history	Invoice/payment writes
Group 4 / CRM-4	Marketing lists and campaign history	Lists, campaign history, contact consent	Outbound marketing delivery in V1
Group 5 / CRM-5	Migration control center and reports	Unified migration dashboard/read model	Duplicate implementations of pipeline engine
Group 6 / ERP-1	Products and inventory	Product master, SKU, stock and warehouse rules	Customer/vendor master records

Workstream	Main responsibility	Owns	Must not duplicate
Group 7 / ERP-2	Vendors and purchasing	Vendor master, purchase orders/lines	Product master or accounting ledger ownership
Group 8 / ERP-3	Sales orders, invoices and payments	Orders, invoices, payment records	Customer master or general ledger ownership
Group 9 / ERP-4	Finance and accounting	Chart of accounts, journal and balances	Domain invoice/order ownership
Group 10 / ERP-5	HR and employees	Employee, department and leave records	Hub credentials/roles or detailed payroll engine
Group 11 / Platform + Migration Kit	Shared engineering foundations and generic migration pipeline	Shared app/tooling/auth primitives, outbox relay, generic migration UI/engine	Per-module copies of shared services
Workstream 12 / PMO, Integration, QA, Release	Contracts, system-level verification and gates	API freeze, integration plan, evidence, go/no-go	A second Compose/CI/telemetry/event relay implementation

4.1 Four module roles

The standardization plan allocates the 30 issue slots per business module by role. Issue number ranges are a standardization convention; use the actual assignee and role field on the live GitHub issue if a correction has been approved.

Role	Slots	Primary responsibility	Typical deliverables
Role 1 - Team Lead / Core Backend	8	Domain logic, repository/query boundaries, business invariants and integration design	Schema/repository, list/detail APIs, mutations, state transitions, relation integrity, events/audit
Role 2 - Integration / DevOps / Adapter	7	Module registration, adapter integration, shared auth use, import receipt/event integration	Configuration, staging mapping, adapter, retries, RBAC declaration, contract integration
Role 3 - Frontend / Review UI	8	Accessible module screens and workflows using shared UI	Data table, filters, states, form/detail edit flow, module configuration for shared migration screens
Role 4 - Data Reliability / QA	7	Data dictionary, dirty-data pack, profiles, rule evidence and regression	ERD/review, synthetic fixtures, profiling tests, contract tests, reconciliation proof, handoff evidence

A role owns an outcome, not every line of code in it. A teammate may help, but the issue owner remains accountable for acceptance evidence and communication. Shared UI/engine screens belong to Team 11; Role 3 provides route/page integration and metadata, not a second migration wizard.

4.2 Team 11 and Workstream 12

- Team 11: `PLT-01..PLT-12` and `KIT-01..KIT-18` = 30 issue slots. It builds shared foundations and the generic engine/UI.
- Workstream 12: `INT-01..INT-06`, `QA-01..QA-08`, `REL-01..REL-04`, plus `PMO-01..PMO-12` = 30 slots. Platform implements common tools; PMO verifies that teams use them and collects evidence.

5. Issue anatomy and how to select the next issue

5.1 Read the entire issue

Before touching code, read the title, purpose/context, exact decision, constraints, out-of-scope section, interface contract, files, acceptance criteria, dependencies and deferred-decision table. Read any linked SDS/ADR and review prior comments. A title alone does not contain the contract.

For every issue, answer in a scratch note:

1. What observable result changes when this issue is complete?
2. Who owns the changed data/API/component?
3. What are the inputs, outputs, error cases and security boundaries?
4. What prerequisite issues/PRs must be merged first?
5. Which acceptance criteria need automated tests, and which need visual/demo evidence?
6. Which files are likely affected, and which files must not change?
7. What could be damaged by a bad implementation or migration?
8. How will a reviewer reproduce and verify the result?

If any answer cannot be found in the issue or approved design, do not invent it. Make a precise question with the relevant field/contract and offer a recommended default to the Team Lead/Integration Lead. Record the resolved answer in the issue before coding. "No open questions" means decisions are documented, not that a developer guesses.

5.2 Check dependencies and readiness

- Confirm every dependency is complete/merged, not merely "in progress."
- Pull the latest base branch and inspect CI state.
- Confirm the issue is assigned to you, has a target sprint and has not been superseded or already implemented.
- Check the Project board and issue comments for another PR or an overlapping branch.
- If blocked, mark the issue blocked using the repo's existing label/status conventions, comment with the blocking issue and exact needed decision, then notify the Team Lead. Never bypass a dependency just to preserve sprint metrics.

5.3 Size and split

The planning standard targets roughly 0.5-2 developer-days for one well-scoped issue; this is a planning guide, not a promise or productivity score. Do not force an issue into the wrong size by producing a giant PR. If work is materially larger, identify distinct independently testable outcomes and ask the Team Lead/PM to approve split issues before implementation. Never create duplicates or trivial filler only to hit the count of 30.

6. The eight-step delivery workflow

Use this sequence for every issue.

Step 1 - Claim and prepare

1. Open the live GitHub issue and confirm assignee/dependencies/status.
2. Comment briefly when work starts if that is your team's norm: `Starting implementation; first pass is repo/contract inspection, then I will post the plan.`
3. Ensure your local tree is clean. Do not mix an old incomplete task into a new issue.
4. Create a branch from the current approved base branch, using the exact issue identifier.

Step 2 - Inspect before editing

Read the repository-level contributor instructions (`README`, `CONTRIBUTING`, `AGENTS.md`, `GEMINI.md`, `.agents/rules/` if present), the relevant package's `package.json`, its exports, adjacent tests, schema/API definitions and recent related PRs. Search for existing helpers first. The shared helper may already solve most of the task.

Do not assume a file path from an AI answer. Search the repository and verify the actual path. Do not create a parallel auth helper, migration engine, event relay or UI component because you did not find the existing one on the first search.

Step 3 - Produce a plan before code

Ask the AI/IDE to explain the issue, list files to inspect/change, identify dependencies, propose tests and flag uncertainties **without modifying files**. Review the plan yourself. If schema/API/event contracts change, get the owning maintainer's approval first.

Step 4 - Implement the smallest end-to-end slice

Make the smallest coherent change that satisfies the issue acceptance criteria. Follow neighboring code style. Add tests at the same time. Do not reformat or refactor unrelated files. Do not edit files owned by another team without agreement, especially shared schema registry, API contracts, component library, package manifests and CI workflows.

Step 5 - Verify incrementally

Run the narrowest relevant test first, then package checks, then broader tests. Inspect every test failure; do not delete or weaken a failing test merely to turn CI green. Verify expected and failure behavior. For UI, run the browser flow and provide screenshots where useful. For data work, inspect counts, outcomes, persisted rows and receipts.

Step 6 - Inspect the diff like a reviewer

Run `git status`, `git diff --check` and `git diff`. Review every changed line. Ask: Is this within scope? Is every added dependency necessary? Is a secret or real dataset present? Did the lockfile change intentionally? Are types being bypassed? Are there unbounded queries, silent catches, duplicated business rules or missing negative tests?

Step 7 - Open a clear PR

Use the PR template. Explain what changed, why, tests and actual outcomes, data/schema impact, risks and how to verify. Include the real GitHub issue reference. Link the issue using `Closes #123` only when the PR should automatically close that specific GitHub issue on merge. An internal ID like `ERP1-01` is not the same as GitHub issue number `#123`.

Step 8 - Review, fix, merge and close

Respond to every review comment. Push updates to the same branch/PR, rerun checks, mark conversations resolved after fixing them, and request re-review when needed. Do not merge until required approvals and status checks pass. After merge, confirm the correct issue closed, update docs/evidence and communicate the result to dependent teams.

Worked example - taking one issue from assignment to PR

Suppose you are assigned `ERP1-01`, the product-schema issue. The catalog says it depends on `PLT-01` and `PLT-03`. This is a workflow example; the live GitHub body remains the exact contract.

1. Open the live `ERP1-01` issue. Read acceptance criteria and confirm `PLT-01` and `PLT-03` are done/merged. If either is incomplete, do not code the dependent issue; report the blocker.
2. Read the ERP SDS and repository instructions. Find the actual Drizzle schema registry, migration location, test factory and related module schema. Do not assume their paths from this guide.
3. Run Prompt A. It should identify constraints such as SKU uniqueness, correct field types/nullability, relationships and migration tests based on the issue/SDS.
4. Run Prompt B and review the file list. If the plan modifies another team's schema registry, root lockfile or shared migration config, get permission/coordination first.
5. Create `feat/ERP1-01-product-schema` from the approved base branch. Implement the approved schema/migration and tests. Do not add API/UI or refactor unrelated modules unless the issue explicitly requires it.
6. Run the repository-defined schema/migration and test checks. Confirm a fresh database setup works, expected constraints are enforced, and no real data or secret is committed.
7. Inspect `git diff` line-by-line. Commit the scoped change, push the branch and open the PR template. Map each schema acceptance criterion to a test/evidence item.
8. Request the correct module reviewer and platform reviewer if a shared contract was touched. Respond to every comment. Merge only after approvals/checks pass; then verify the live issue status.

The same sequence applies to a UI issue, migration adapter, event producer or QA issue. The test evidence changes by issue, not the discipline of the workflow.

7. Git and GitHub: the working rules

7.1 Concepts in one minute

- **Repository:** the project's version-controlled files and history.
- **Issue:** the agreed unit of work, with acceptance criteria and ownership.
- **Branch:** an isolated line of work; never work directly on `main`.
- **Commit:** a named snapshot of changes.
- **Pull Request (PR):** proposal to integrate a branch into the base branch, including review and CI.
- **Review:** a human checks logic, tests, security, scope and maintainability.
- **CI/status checks:** automatic tests/format/type/build checks run against a change.
- **Merge:** the approved integration of the PR. A PR being open or approved is not the same as merged.
- **Milestone:** groups issues/PRs toward a delivery gate; labels classify items.

Use the repository's actual branch policy. The recommended baseline is protected default branch, PR-only changes, required CI checks, at least one reviewer other than the author, and resolved conversations before merge. Administrators must configure enforcement; this guide cannot prove the live repository has those settings. GitHub documents branch protection and required checks [3].

7.2 Labels and status

Use existing labels and the team Project board rather than inventing near-duplicates. At minimum, people need to distinguish:

- priority (`P0`, `P1`);
- area (`erp1`, `crm3`, `platform`, `migration-kit`, `integration`, `qa`, `release`);
- type (`feature`, `bug`, `test`, `docs`, `architecture`, if those are the repo's approved names);
- blocked / ready / in progress / in review / done status in the Project workflow.

The issue body is the specification. Labels and status help you find it; they do not replace its content. GitHub's issue quickstart describes assignment, labels, milestones and Projects [1].

7.3 Branch naming

Use one branch per issue, from the latest approved base branch:

```
# Confirm current location and clean state
git status --short --branch

# Refresh remote metadata
git fetch origin

# Substitute the repository's actual base branch if it is not main
git switch main
git pull --ff-only origin main

# Example only: preserve the issue ID exactly
git switch -c feat/ERP1-01-product-schema
```

Suggested prefixes: `feat/`, `fix/`, `test/`, `docs/`, `chore/`. Keep names short and include the issue ID. Examples: `feat/CRM2-06-deal-state`, `fix/KIT-03-csv-profile-null-count`, `test/QA-04-load-retry`.

Never run `git push --force`, `git reset --hard`, delete branches, rewrite history or run destructive database commands unless an authorized maintainer has explicitly approved the action and you understand the consequence. If you have uncommitted work from another task, stop and ask before switching branches.

7.4 Commits

Make commits small and understandable. Use the project's configured hooks and commit conventions. Suggested format if the repo has no convention:

```
feat(erp1): add product schema and constraints
fix(kit): reuse idempotency key on retry
test(crm3): cover illegal ticket transitions
docs(team): clarify PR evidence requirements
```

Do not commit `.env`, credentials, API keys, personal datasets, `node_modules`, build output, local databases or Antigravity scratch artifacts. Never force-add files ignored by `.gitignore` without maintainers approval.

7.5 Issue ID vs GitHub issue number

Keep the permanent catalog ID (for example `ERP1-01`) in branch name, PR title and text. Use the actual GitHub issue number for automatic closure syntax, for example `Closes #234`. If a GitHub issue does not yet exist or the PR should not close it, use a non-closing reference such as `Refs ERP1-01` or link the issue URL. Do not fake issue numbers.

8. Local environment and repository discovery

The repository is the authority for exact commands, package names, environment variables and file locations. Do not paste commands from an old guide blindly.

8.1 First inspection

```
git status --short --branch
git remote -v
node --version
pnpm --version

# Inspect root scripts and workspace config; do not assume script names
cat package.json
cat pnpm-workspace.yaml
cat turbo.json
pnpm run
```

If an optional file does not exist, do not create one just to satisfy a command. Inspect the repo README and team-specific setup instructions first.

8.2 Typical setup sequence

Use the repository's documented equivalent. These commands are examples for the approved stack, not proof that every file/compose service exists:

```
# From the cloned repository root
corepack enable
pnpm install --frozen-lockfile

# Only if .env.example is part of the repo; edit the local copy with safe local values
cp .env.example .env

# Only if the repository provides a root compose file and documented services
docker compose config
docker compose up -d --build
```

Never paste real credentials into a prompt, issue, commit, fixture, screenshot or log. Local `.env` files must remain untracked. Prefer `.env.example` with placeholder values and a documented configuration schema.

8.3 Discover exact check commands

Read root and package `package.json` scripts. Run what exists, in the documented order:

1. formatter/lint check;
2. TypeScript type-check;
3. unit/domain tests;
4. API and contract tests;
5. relevant frontend component tests;

6. Playwright journey for UI changes;
7. build;
8. k6 performance check only when the issue requires it.

Use the exact scripts defined by the repository. Do not invent `pnpm test`, `pnpm lint` or `pnpm build` if the project uses different names. Capture the command and its real exit result in the PR.

8.4 If setup fails

Copy the exact error, the command, runtime/tool versions and a minimal relevant log excerpt. Redact secrets and personal data. Check lockfile/runtime mismatch, required services, missing env variables and whether another process already owns the port. Ask AI to explain options; do not accept a broad cleanup command without understanding what it deletes.

9. Using AI effectively without surrendering engineering judgment

AI is a fast junior pair-programmer and reviewer. It can draft plans and tests, search unfamiliar code, explain errors, and suggest implementations. It can also hallucinate paths, introduce dependencies, miss edge cases, confidently misread contracts or change unrelated code. The human contributor owns the result.

9.1 Use AI in stages

1. **Understand:** Ask it to summarize the issue and acceptance criteria. No edits yet.
2. **Discover:** Ask it to find the relevant existing implementation, owners, tests and nearby patterns. Require real repository paths and quotes/snippets for claims.
3. **Plan:** Ask for a small plan and file list, tests, risks and unknowns. Review it before allowing edits.
4. **Implement:** Give one well-bounded change at a time. State constraints and explicitly forbid unrelated refactors/dependencies.
5. **Test:** Ask for unit, negative, contract and edge-case tests. Run them yourself.
6. **Critique:** Ask a separate review pass to inspect the diff for correctness, security, data integrity, races, scope creep and missing tests.
7. **Explain:** Ask the agent to explain the final diff in language you understand. If you cannot explain a changed function, do not approve your own code.

9.2 Minimum context to give the AI

Include:

- the full live GitHub issue body, not only its title;
- actual issue ID and dependency status;
- relevant files and contract/ADR references;
- applicable stack rules and explicit out-of-scope items;
- available scripts/commands and environment constraints;
- acceptance criteria and required evidence;
- instruction to inspect before editing and ask if the specification genuinely conflicts.

Do not dump the entire repository into one prompt. Start with the issue, allow targeted searches, and bring in only relevant files. Keep one primary task per agent conversation.

9.3 Prompt rules that improve results

Good prompt: **one objective + source of truth + constraints + expected output + tests + stop conditions**.

Bad prompt: “Build the whole module, make it perfect, fix anything else, and improve the architecture.” The scope is not reviewable and the agent may rewrite working code.

Require the agent to report:

- files read;
- files changed;
- assumptions made (ideally none not already approved);
- commands executed and true results;
- tests added and which acceptance criteria they prove;

- known gaps that block completion.

"Tests pass" is not evidence unless the command ran and exited successfully. A code-generated test is not enough if it merely restates the implementation or does not test failure paths.

10. Google Antigravity: safe, repeatable workflow

Antigravity can work across the editor, terminal and browser, and can produce implementation plans, code diffs and other artifacts [4][5]. Treat these capabilities as tools that need review, not as authority to alter the repository unchecked.

10.1 Before the first task

1. Open the correct cloned ClearShift repository/workspace root.
2. Confirm the Git branch and working tree are the ones you intend to change.
3. Read the repo's `AGENTS.md`, `GEMINI.md` and `.agents/rules/` instructions if present.
4. Check the active project/workspace and permission settings.
5. Start with a planning/analysis request and review the artifact/plan before allowing edits.
6. Keep terminal/file permissions conservative. Do not use an unrestricted mode for convenience. Antigravity's permission behavior differs by OS and configuration; consult the official permission guide for the current UI [6].

10.2 Use Rules for permanent constraints, Skills for repeatable tasks

- Put project invariants in a workspace rule (for example the included `.agents/rules/clearshift-core.md`): approved stack, no real data, no unapproved queue, no cross-module ownership violations, tests required.
- Put a repeatable multi-step process in a skill (included at `.agents/skills/clearshift-issue-execution/SKILL.md`): inspect issue, plan, implement, test, review diff, prepare PR.
- Do not rely on old Antigravity Workflows for new onboarding. The official migration guidance says legacy Workflows cease being indexed/executable on 19 October 2026 and recommends migrating them to Agent Skills [7]. Use the actual controls available in your installed version.

10.3 How to use the agent panel

Ask for one phase at a time:

Analysis phase: "Do not edit. Inspect the issue and repository; return a proposed plan, files to read/change, relevant existing helpers, dependencies, risks and test commands found in package scripts."

Approval phase: Read the plan. Correct any wrong owner, invented file path, architecture drift or out-of-scope change. Approve only the bounded plan.

Implementation phase: "Implement only the approved plan. Stop and ask before changing shared contracts, dependency versions, database ownership, CI, auth, outbox or files owned by another team."

Verification phase: Ask Antigravity to run permitted checks and report actual results. Inspect terminal output and diffs. Run critical checks yourself when possible. Never tell the PR that a test passed merely because the agent said it did.

Walkthrough phase: Ask for a concise diff walkthrough: each file, behavior changed, test proving it and any risks. This is also a test of whether you understand your own patch.

10.4 Parallel agents and worktrees

Parallel work is safe only when ownership and file boundaries are explicit. Do not let two agents edit the same files or shared schema/API contract concurrently. For parallel investigation, delegate read-only analysis (test inventory, API contract inspection, security review). For parallel code, use separate branches/worktrees and coordinate interfaces with owners first. A final human integrates and reviews every diff.

10.5 Never approve these blindly

- deleting or rewriting databases, migrations, branches or Git history;
- `rm -rf`, `git reset --hard`, force pushes or mass file deletion;
- changing queue libraries, runtime/framework major versions or lockfiles;
- widening permissions, uploading files or navigating authenticated pages;

- changing JWT/RBAC, secrets handling or sensitive-field serialization;
- weakening/removing tests to get CI green;
- auto-merging uncertain duplicates or inventing business defaults.

11. Implementation standards by technical area

11.1 Database and Drizzle

- Read the current schema registry and adjacent migrations before adding entities.
- Use UUID primary keys where specified, typed columns, explicit nullability, unique/check/foreign-key constraints and useful indexes.
- Put durable business invariants in database constraints where feasible, then mirror validation at the API/domain boundary for helpful error messages.
- Create forward migrations using the repo's migration system. Do not rewrite an already-applied migration. Do not drop/rename data-bearing columns casually.
- For every migration, test both a clean database migration and the expected upgrade path. If rollback is unsupported, state that and provide a recovery approach; never claim automatic rollback without a tested design.
- Use transactions where an operation must be all-or-nothing locally. Target business row plus target import receipt must commit in the same PostgreSQL transaction.
- Avoid unbounded list queries. Bound pagination and use deterministic sorting with a stable tie-breaker.

11.2 Fastify APIs, Zod and OpenAPI

- Treat approved OpenAPI as a contract. Add schemas for request params, body and response; Zod validation should match API errors and documented response shapes.
- Follow the existing common error envelope and response codes. Do not add a one-off error format.
- For list endpoints use bounded pagination, parameterized filters and allowlisted sort fields. Reject malformed pagination rather than silently executing an unbounded query.
- Define whether a field is mutable, computed, server-owned or read-only. Do not trust caller-supplied computed totals, Golden IDs or actor IDs.
- Test both success and failures: invalid body, missing resource, unauthorized role, duplicate unique key, invalid state transition and missing relationship.
- If a consumer needs data owned by another module, call its API or approved projection. Do not import its internal repository or mutate its tables directly.

11.3 Frontend React + TypeScript + Vite

- Use shared shell, design tokens and approved components; do not create per-team versions of shared buttons, forms, tables or migration wizard.
- Keep network state explicit: loading, success-with-data, empty, validation error, server error and retryable failure.
- Provide accessible labels, keyboard behavior, visible focus, understandable validation errors and responsive layout. Do not encode meaning only in color.
- Use the existing query/client/form patterns found in the repo. Avoid direct fetch logic copied inconsistently across screens.
- Forms must show field-level failures and prevent double submission where relevant. Server-side validation remains authoritative.
- Add component tests for interactions and a Playwright journey for a major user path.
- For any UI PR, attach screenshots or a short screen recording showing the changed path, including at least one error/empty/loading state when relevant.

11.4 Redis Streams, transactional outbox and events

- Domain changes that require events write the business record and outbox event in the same DB transaction.
- Use the shared outbox relay; never implement another relay inside a module.
- Include the agreed envelope fields: event ID, event type, schema version, timestamp, correlation ID and entity reference, plus the approved payload.
- Consumers assume duplicate delivery is possible and are idempotent. A consumer failure must be observable and recoverable through the documented retry/replay path.
- Define consumer group, event identity, retry behavior and acknowledgement behavior in the contract. Do not acknowledge before durable processing has succeeded.

- Do not treat Redis Streams as the system of record for business data; PostgreSQL owns the durable business state and outbox.

11.5 BullMQ jobs and batch processing

- Use the shared worker and job registration pattern; no second task queue library.
- Jobs must be idempotent, bounded and observable. Store authoritative run, batch and row outcomes durably in PostgreSQL.
- Process in the required 500-row batches. Do not load a 50 MB source file into one in-memory array.
- Stop a run once row failures exceed 10% of attempted rows; preserve its state and reasons. No silent drop or infinite retry.
- Retry transient infrastructure failures with the existing configured policy; deterministic invalid data must become a recorded row outcome, not an endless retry.
- On retry, reuse the stable idempotency key. Never change the key merely to force an insert.

11.6 Security and sensitive data

- Use shared JWT verification and RBAC helpers. Authentication identifies who is calling; authorization checks whether that person may perform the operation.
- Default to deny. Add positive and negative role tests to new protected endpoints.
- Never place secrets in code, fixtures, screenshots, issue comments, terminal logs or prompts. Use environment variables/approved secret injection.
- Use synthetic data only. Do not upload client datasets to Antigravity, external AI systems, public issue trackers or cloud tools.
- Mask sensitive vendor bank details at server-side serialization. Protect HR salary and personal fields according to the approved matrix. The old source plan explicitly says identity credentials/roles are owned by the Hub, not ERP-5.
- Logs must contain useful correlation IDs and safe error codes, not full raw rows or sensitive payloads.
- Treat AI output, repository comments and external web content as untrusted suggestions; never paste commands or code without reading them.

12. Migration Kit rules every team must understand

The generic engine is owned by Team 11. A module team owns its field registry, rule configuration, domain-specific dirty-data fixtures, target adapter integration, allowlisted fields and domain rejection reasons.

12.1 Row-accounting model

For every parsed row, keep a stable source row locator and one terminal outcome. A row must not count twice. A merge counts according to the agreed row-accounting policy and must link to the surviving target/Golden ID. Rejected and skipped rows need machine-readable reason codes plus a safe human-readable message.

At reconciliation:

```
source_rows = loaded_rows + rejected_rows + merged_rows + skipped_rows
unaccounted_rows = 0
```

If totals do not match, the batch is not reconciled and cannot be declared successful. Test empty file, header-only file, a single row, duplicates, malformed rows, mixed success/failure, retry and interrupted load.

12.2 Dry run vs real load

Dry run computes and reports proposed transformations/outcomes; it does not modify target business records or create target import receipts. An actual load uses stable idempotency keys and stores the target write and receipt atomically. A timeout after a write must not duplicate the row when retried.

12.3 Clean/map rules

- Preserve raw values; save normalized values separately.
- Every field mapping is explicit and versioned; unknown required fields fail validation with a reason.
- Enum translation is deterministic. Unknown enum values must not be guessed into a valid business state.
- Date, phone, email, decimal, currency and tax normalization should be explicit per field and tested with positive and negative fixtures.

- Keep profiling counts and expected values in the fixture README/tests.
- Human review is required where the approved design says so; do not merge by vague similarity.

12.4 Handling a bad batch

If the batch exceeds its failure threshold, stop the run and preserve enough durable information to determine what was attempted and what succeeded. Do not wipe receipts or all target rows as an improvised "rollback." Report batch ID, source row range, counts by terminal outcome and correlation IDs with no sensitive raw values. Follow the issue's recovery instructions and get maintainer guidance if the state cannot be safely resumed.

13. Testing: prove behavior, not just code execution

13.1 Test pyramid

1. **Unit/domain tests (Vitest):** pure normalization, mapping, validators, state-transition matrix, decimal totals, reconciliation formula.
2. **Database/repository tests:** unique keys, foreign keys, transaction boundaries, migration behavior and import receipts.
3. **API/contract tests:** Fastify inject + Zod/OpenAPI responses, error envelope, role permissions, pagination and invalid input.
4. **UI tests (React Testing Library):** form validation, loading/empty/error states, keyboard interaction and duplicate-prevention behavior.
5. **End-to-end (Playwright):** main user journey from source intake through review, dry run, load, validation and reconciliation; module journeys for create/edit/detail where applicable.
6. **Performance (k6):** only where the issue/test plan sets workload and pass threshold. Record test data, command, environment, concurrency and result.

13.2 Required negative cases

For a typical module API, test missing ID, invalid body, duplicate unique value, missing foreign key, forbidden role and illegal state change. For migration code, test malformed encoding/row, missing required field, duplicate, idempotent retry, timeout/restart, threshold stop, rejected row reason and exact reconciliation. For event consumers, test duplicate delivery and replay. For finance, test an unbalanced journal, closed period and concurrent posting boundary.

13.3 Reconciliation acceptance

Use known fixtures with predetermined expected counts, not random data with no oracle. The QA owner should be able to state before running the test: source rows, expected loaded/rejected/merged/skipped counts and why. The test fails if `unaccounted != 0` or any row has zero or multiple terminal outcomes.

13.4 Evidence to attach to a PR

- actual test commands and pass/fail results;
- relevant unit/API/UI/e2e test files;
- UI screenshots or a concise recording for visual changes;
- migration/ERD/API/event contract diff where applicable;
- fixture counts and reconciliation report for data work;
- security/permission negative tests for protected paths;
- k6 output only if the issue requires performance verification;
- known limitations and linked follow-up issues (not hidden TODOs).

14. Creating a high-quality pull request

A PR is a technical review packet. Reviewers should not have to guess what you changed, how to run it or whether it is safe.

14.1 Keep the PR reviewable

- Prefer one PR per issue and one logical outcome.
- Keep unrelated refactors, formatting sweeps and package upgrades out.

- Include tests that prove the acceptance criteria.
- Generated code is not exempt from review.
- If a PR needs broad scope expansion, pause and ask the issue owner/PM to split it or approve the new scope.
- Never self-merge around failed CI or unresolved review.

14.2 Before opening the PR

```
git status --short --branch
git diff --check
git diff --stat
git diff
```

Then run all applicable commands discovered in `package.json`. Inspect untracked files as carefully as tracked files. Confirm no secrets, local data, debug logs or accidental lockfile churn. Ensure the branch is based on the correct current target branch according to repo policy.

14.3 PR title

Suggested format:

```
[ERP1-01] feat(erp1): add product schema and constraints
[CRM3-06] fix(crm3): reject invalid ticket state changes
[KIT-03] feat(kit): profile missing values in streamed CSV rows
[QA-04] test(qa): cover interrupted batch recovery
```

Use the repository's required title convention if one exists. Keep the permanent issue ID visible.

14.4 PR body must answer

1. **Issue:** link to the real GitHub issue number and keep the catalog ID.
2. **What changed:** behavior, files and important design choices.
3. **Why:** map each change to an acceptance criterion.
4. **Tests:** commands actually run and actual outcomes; include what was not run and why.
5. **Data/API effects:** schema migration, contract changes, event changes, fixture impact and compatibility notes.
6. **Security:** auth/RBAC, sensitive data, secrets/logging, upload behavior as applicable.
7. **Visual evidence:** screenshots/recording for UI.
8. **Risk/recovery:** failure mode and safe recovery, particularly for batch/load code.
9. **Out of scope:** explicitly state major adjacent work not changed when reviewers could expect it.

14.5 Link the issue correctly

Only use `Closes #123` when `123` is the live GitHub issue number that the PR completes. If a PR fixes several issue IDs, prefer separate PRs; where multiple issues truly form one indivisible change, link every actual issue number and explain the dependency to the reviewer. An internal catalog code such as `KIT-03` does not automatically close an issue without the actual GitHub number.

14.6 Reviewer assignment and merge

Request the team lead or relevant code owner, and an integration/platform reviewer when touching shared contracts. Do not request approval from a person who did not review the code. Wait for required checks and approvals. Address comments in code, push commits to the same branch, and resolve review conversations only after the change is made. GitHub's PR quickstart documents requesting review, addressing feedback and merging after checks pass [2].

14.7 Push and create the PR - exact sequence

After checks have run and the diff is reviewed, stage only intended files. Replace the example branch, commit message and file paths with yours.

```
# Confirm branch and all changed/untracked files
git status --short --branch
```

```
# Final whitespace/conflict-marker check
git diff --check

# Inspect the staged/unstaged diff before committing
git diff --stat
git diff

# Stage only the intended paths; do not blindly use `git add .`
git add path/to/file.ts path/to/test.test.ts

git diff --cached --check
git diff --cached

git commit -m "feat(erp1): add product schema and constraints"

# Push your branch; the first push sets its upstream
git push -u origin feat/ERP1-01-product-schema
```

Then in GitHub:

1. Open the repository and select **Compare & pull request** for your branch (or Pull requests -> New pull request).
2. Check the **base branch** against repo policy, and the **compare branch** is yours. Do not assume `main` if the repo uses another integration branch.
3. Paste a descriptive title using the issue ID; complete every applicable section in `.github/PULL_REQUEST_TEMPLATE.md`.
4. Link the actual live issue. Use `Closes #<number>` only with its real GitHub number and only if merging should close it.
5. Add relevant reviewers and existing labels/project fields; choose Draft if evidence is not complete. A Draft PR is for early discussion, not a shortcut around required checks.
6. Review the rendered body for code fences, screenshots and links. Submit/mark ready only when it is reviewable.
7. Watch CI and reviewer feedback. Fix the source branch, push again, and make sure the new commit reruns relevant checks. Do not use a second PR just to avoid a failing check.

Optional GitHub CLI

Use these only if `gh` is installed, authenticated and allowed by the repository maintainers. UI use is fine if you are new to the CLI.

```
gh auth status
gh pr status
# Create a draft only after branch is pushed and PR body is complete:
gh pr create --draft --base main \
  --head feat/ERP1-01-product-schema \
  --title "[ERP1-01] feat(erp1): add product schema and constraints" \
  --body-file /path/to/pr-body.md
```

The branch, base, title, body path and issue number are examples. Verify each before running; don't paste example placeholders into production commands.

15. Reviewing another person's pull request

Do not approve by checking that the code "looks fine." Review against the issue and its tests.

Reviewer checklist

- ☐ Does the PR solve the linked issue and no larger unrelated problem?
- ☐ Does every acceptance criterion map to code or evidence?
- ☐ Are shared contracts and module data ownership respected?
- ☐ Are success, validation, missing-resource, forbidden and failure paths handled?
- ☐ Are transactions, retries, idempotency and concurrency safe for the operation?
- ☐ Are raw and sensitive values protected in logs/responses/exports?
- ☐ Is there a test that would fail if the key rule were broken?

☐ Are tests meaningful (not snapshots or mocks that bypass the logic under test)?
☐ Are database migrations forward-safe and tested?
☐ Does the UI handle loading, empty, error, keyboard and validation states?
☐ Are docs, API schemas, data dictionaries, fixtures and screenshots updated where needed?
☐ Did the author run and report the relevant commands honestly?
☐ Are all inline conversations resolved and CI green?

If you do not understand the code, ask for an explanation or test it locally. You are accountable for your approval.

16. Debugging, conflicts and recovery

16.1 When a test fails

1. Reproduce it with the narrowest command.
2. Read the first relevant stack trace and the actual assertion, not only the last red line.
3. Determine if the failure is in implementation, test data, environment or contract.
4. Write a small failing test/reproduction before broad code edits where possible.
5. Ask AI for likely causes and a diagnostic plan; require it not to edit yet.
6. Make one fix at a time; rerun the failing test then the broader suite.
7. Report a real blocker with logs redacted if it is an environment/service issue.

Do not suppress errors, add broad retries around deterministic failures, or weaken assertions without approval.

16.2 When merge conflicts happen

- Stop and preserve your work. Check `git status` before any command.
- Fetch the latest base branch and inspect which files conflict.
- For a shared contract/schema conflict, involve both owners. Do not resolve a semantic conflict by choosing "ours" or "theirs" without understanding both changes.
- Resolve the conflict, run formatting/types/tests that cover the conflict and inspect the complete diff.
- Commit the resolution and update the PR. If unsure, ask the branch owner/maintainer before continuing.

16.3 When you discover a design flaw

Do not quietly redefine the specification in code. Comment on the issue with:

- the exact requirement/line that conflicts;
- a minimal reproducible example or concrete scenario;
- the risk of following it literally;
- a recommended option and alternative;
- tests/contracts/documents affected;
- who must approve the decision (module lead and, for shared boundaries, Integration Lead/PM).

Wait for a written answer. Then update the issue/ADR/docs and only then implement.

17. Daily work, sprint routines and escalation

Daily update (2-3 minutes)

Report: (1) issue/PR moved, (2) exact result, (3) today's next step, (4) blocker and the person/decision needed. "Working on backend" is not a useful status. "ERP2-03 create/update API is implemented; invalid vendor ID returns 400; waiting for PLT-04 error-envelope merge" is useful.

Meetings and coordination

Use the project plan's recurring rhythms: group stand-up daily, leads meeting weekly, Integration Clinic twice a week for Role 2/integration issues, QA sync weekly, sprint demo and retrospective at sprint end. Keep important decisions in the GitHub issue/ADR/decision log, not only in chat.

Escalation rules

Escalate the same working day when:

- an issue is blocked by a dependency for more than one working day;

- an API/schema/event contract conflicts across two teams;
- a shared file is being changed concurrently;
- a P0 defect, secret exposure, data loss or reconciliation mismatch appears;
- AI suggests an architecture change outside the approved baseline;
- a test or deployment command may destroy data or rewrite shared history.

For security exposure or possible data loss, stop the affected operation first; inform the team lead and maintainers immediately. Do not wait for the next stand-up.

18. Module-specific cheat sheets

These are reminders, not substitutes for the detailed issue/SDS. The owning issue defines the full acceptance criteria.

Module	Main concepts	Rules to actively test
CRM-1 Contacts, Companies, Leads	Customer master data, company/contact links, lead state, Golden ID	Unique/normalized identifiers; uncertain matches go to human review; never overwrite unsubscribe/consent with a permissive value
CRM-2 Deals, Pipeline, Quotes	Deal owner, stages, quote value and product references	Legal stage transitions; read product/pricing data from ERP-1; a won-deal event must be idempotent
CRM-3 Support Tickets and Activities	Ticket/customer link, priority, owner, status, activity history	Cannot reference missing customer; invalid transitions rejected; linked invoice/order status is read through the owning API
CRM-4 Marketing Lists and Campaign History	List membership, recorded campaign history, consent/unsubscribe	V1 records history; it does not send real email/SMS. A “no contact” consent state must never be overwritten with “yes” by import
CRM-5 Migration Control Center	Batch status, counts, errors, event-driven dashboard/reporting	Read/report across modules using contracts; dashboard counts must agree with durable ledger/reconciliation, not a stale browser cache
ERP-1 Products and Inventory	SKU, category, unit, decimal price, tax rate, one warehouse, stock level	SKU uniqueness; units from allowlist; prices/tax are validated; no negative stock; opening stock uses migration runner path
ERP-2 Vendors and Purchasing	Vendor, purchase order, PO lines, status	Mask vendor bank details server-side; lines reference known products; total is computed with decimal arithmetic and documented rounding
ERP-3 Sales Orders and Invoices	Customer Golden ID, order lines, invoice number/status, payments	Must reference customer/product; due date required to determine overdue status; use NUMERIC/decimal for money; payment and invoice totals reconcile
ERP-4 Finance and Accounting	Chart of accounts, journal entry/lines, opening balances, trial balance	Debit must equal credit; unbalanced journal rejected; closed month rejects posting; opening balance loaded once; concurrent posting must be safe

Module	Main concepts	Rules to actively test
ERP-5 HR and Employees	Employee, department, leave; Hub identity projection	Hub owns credentials/roles; salary and personal details are protected; no real personal data; only approved payroll aggregate per department/month
Platform / Migration Kit	Shared contracts, auth, shared UI, profiling, mapping, row ledger, batch workers, outbox	Do not fork shared engine; stable idempotency, row-level terminal outcomes, dry-run immutability, bounded batch failure and zero unaccounted rows

Key cross-module master data boundaries

Master data	Owner	Consumers must
Customers/contacts/companies	CRM-1	Read from CRM-1 API/approved projection; use Golden ID mapping
Products/prices	ERP-1	Read products/prices; do not clone mutable product truth
Vendors	ERP-2	Read vendor IDs from ERP-2
Sales order/invoice/payment	ERP-3	Expose state for support/finance by approved contract; avoid direct table writes from other modules
Chart of accounts/journals	ERP-4	Use the chart/account API contract and finance rules
Employees and user roles	ERP-5 owns employee records; Hub owns identity users/roles	Never store credentials in ERP-5 or create shadow roles
Golden ID map and shared event contract	Hub / Platform with Integration Lead	Use the shared service and schema; no local duplicate ID authority

19. Ready-to-copy AI prompts

Copy the prompts in `AI_Prompt_Playbook.md` and replace placeholders. The most important prompts are summarized here.

Prompt A - Understand; do not edit

```
You are my senior engineer for ClearShift. I am assigned GitHub issue [ISSUE
ID + URL].
Read the complete issue, linked SDS/ADR, repository instructions, relevant
existing code and tests.
Do not edit files or run destructive commands.
Return: (1) acceptance-criteria checklist, (2) dependencies and whether they
are complete, (3) existing files/helpers/contracts to reuse with real paths,
(4) likely files to change and files to avoid, (5) tests needed for every
criterion, (6) risks/security/data-integrity checks, (7) unknowns that require
maintainer approval.
Do not invent paths, scripts, API contracts or business rules. If evidence is
missing, say what you searched.
```

Prompt B - Approved implementation plan

```
Based only on [ISSUE ID + URL] and the approved ClearShift design, propose the
smallest implementation plan.
For each change, give file path, reason, acceptance criterion, test and
rollback/recovery consideration.
Preserve the pinned stack and ownership boundaries. Do not change
dependencies, shared contracts, CI, auth, database ownership or queue
```

```
architecture without approval.
Do not edit yet. List blockers before proposing code.
```

Prompt C - Implement one scoped issue

```
Implement only the approved plan for [ISSUE ID]. Keep the diff minimal and
readable.
Follow existing repo patterns and shared helpers. Add tests for normal
behavior, invalid input, permission failures and relevant retries/edge cases.
Do not change unrelated files, weaken tests, introduce dependencies, use real
data, expose secrets or claim tests passed unless you ran them.
Stop and ask before a shared contract/schema decision, cross-team file change
or requirement conflict.
When done, report files changed, behavior, tests run with actual results,
acceptance criteria evidence and remaining blockers.
```

Prompt D - Independent review of my diff

```
Review the current branch against issue [ISSUE ID + URL]. Do not edit files.
Inspect the full diff and tests. Prioritize correctness, security/privacy,
transaction/idempotency, event handling, race conditions, performance, API
compatibility and missing failure tests.
Report findings by severity with exact file/line and a reproducible scenario.
Distinguish confirmed bugs from suggestions. Check for out-of-scope edits,
false test claims, secrets, unbounded data access and violations of ClearShift
V1 boundaries.
```

Prompt E - Prepare the PR packet

```
Using the issue and actual git diff, draft the PR title and body. Do not make
up tests or results.
Map every acceptance criterion to code/test/evidence. Include issue ID and
actual GitHub issue-number closure syntax only if known, commands actually run
and outcomes, schema/API/event changes, screenshots if relevant,
security/privacy review, risks and omitted checks.
Flag missing evidence instead of pretending the PR is ready.
```

20. Checklists and templates

Contributor checklist before coding

- ☐ Full issue read; acceptance criteria understood.
- ☐ Dependencies are merged/done.
- ☐ Repository and package instructions read.
- ☐ Working tree clean and branch contains issue ID.
- ☐ Existing pattern/helper/test found before reimplementation.
- ☐ AI plan reviewed; no unresolved business decision guessed.
- ☐ Out-of-scope boundaries understood.

Contributor checklist before PR

- ☐ Implementation maps to acceptance criteria only.
- ☐ Unit/domain tests added for new logic.
- ☐ Failure/negative cases tested.
- ☐ Data migrations/contracts/docs/fixtures updated as needed.
- ☐ Relevant lint, types, tests and build commands actually ran and passed.
- ☐ UI screenshots or reconciliation evidence attached when relevant.
- ☐ `git diff --check`, `git status` and full diff reviewed.
- ☐ No secrets, real data, unrelated edits or false claims.
- ☐ PR title/body complete and references correct real GitHub issue.

Maintainer checklist before merge

- ☐ Scope matches issue and no undocumented architecture decision is buried in code.
- ☐ Required reviewer approvals and CI checks are green.
- ☐ Acceptance evidence is present and meaningful.
- ☐ No unresolved review conversation.
- ☐ Changes respect module/shared ownership.
- ☐ Security, data migration, retry/idempotency and compatibility risks addressed.
- ☐ PR links the correct live issue; merge method matches repo policy.

Mini status comment template

```
Update - [date]
Issue: [catalog ID + GitHub URL]
Completed: [specific observable work]
Evidence: [tests/PR/contract/test report]
Next: [next concrete action]
Blocked by: [exact dependency/decision/person, or None]
Risk: [known risk, or None]
```

21. Common failure patterns

Failure	Why it is harmful	Correct action
Coding before reading dependencies	Creates duplicate/conflicting interfaces	Verify dependencies and contract first
Asking AI to “build everything”	Unreviewable diff, missed criteria, scope creep	One issue and one bounded plan at a time
Accepting AI paths without checking	Files or helpers may not exist	Search repo and cite real paths
Multiple people/agents editing same shared files	Merge conflict and inconsistent contracts	Agree ownership or separate worktrees and coordinate first
Direct push to main	Bypasses review and CI	Branch + PR + checks
Giant PR for multiple unrelated issues	Review misses behavior; rollback/review hard	One independently reviewable outcome per PR
Changing queue/framework “because easier”	Breaks system-wide architecture	ADR and Integration Lead approval before change
Reusing a retry with a new idempotency key	Duplicate target records	Preserve stable key from the original source row
Treating a dry run as a real import	Data changes during preview	Dry-run must not mutate business rows or target receipts
Ignoring rejected rows in totals	Data loss hidden by reports	Every row gets exactly one terminal outcome
Marking test passed without running it	False confidence and bad gate decision	Report real command and exit/result
Fixing CI by weakening assertions	Hides defects	Fix code or get approval to change the specification/test
Putting sensitive data in AI prompt/fixture	Privacy/security exposure	Use synthetic/minimal redacted data only
Closing issue before merge	Board lies about delivery	Close through correct PR merge/issue workflow
Calling V1 “rollback” a V1 feature	Contradicts the release boundary	Stop safely, preserve evidence, bounded recovery; rollback saga is V2

22. First-day checklist

1. Confirm your team, role, module, team lead and code reviewer.
2. Get the repository URL and read setup instructions.
3. Verify Node/pnpm and the exact pinned project versions.

4. Run the setup and existing baseline checks; record what passes before making changes.
5. Read the module SDS, project plan and shared architecture rules relevant to your first issue.
6. In GitHub, find your assigned issue, read dependencies and acceptance criteria, and add it to the correct Project status.
7. Create a branch with the catalog ID and inspect existing code/tests.
8. Use Prompt A and Prompt B; ask the team lead about any unresolved design decision.
9. Implement one scoped change, test it, open a PR with evidence, and ask for review.
10. Learn by reviewing one merged PR from your module and one shared platform PR.

Closing principle: GitHub defines the work; the IDE changes the repository; AI assists; tests prove behavior; human-reviewed PRs integrate it safely.

23. Reference links

The following official docs support the mechanics in this guide. UI labels may vary by product version; use the repository's contributor instructions first.

1. GitHub Docs - Quickstart for GitHub Issues: https://docs.github.com/en/issues/tracking-your-work-with-issues/learning-about-issues/quickstart
2. GitHub Docs - Quickstart for pull requests: https://docs.github.com/en/pull-requests/get-started/pull-request-quickstart
3. GitHub Docs - About protected branches: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches
4. Google Antigravity - IDE overview: https://www.antigravity.google/docs/ide/overview/
5. Google Antigravity - Artifacts: https://www.antigravity.google/docs/artifacts
6. Google Antigravity - Agent permissions: https://antigravity.google/docs/agent-permissions
7. Google Antigravity - Workflows to Skills migration: https://antigravity.google/docs/migration/workflows-to-skills
8. Google Antigravity - Rules: https://www.antigravity.google/docs/rules/